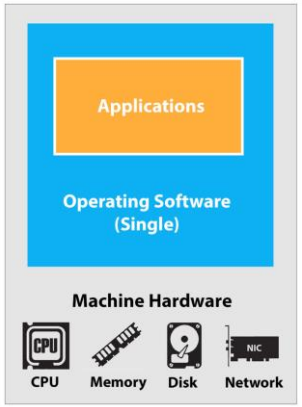


MLOps con Docker

Virtualización

Las computadoras que utilizamos, en general, son “físicas” y ejecutan un único sistema operativo en simultáneo, con el que interactuamos.



Virtualización

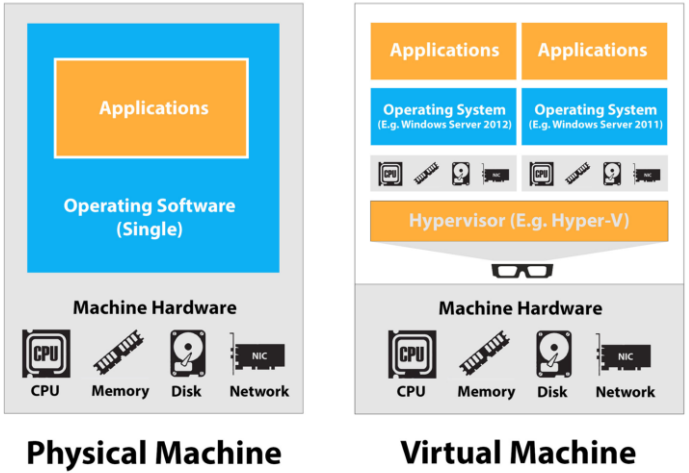
El inconveniente es que durante la mayor parte del tiempo existe una porción de recursos/hardware que no está en uso.

Esto en computadoras de uso personal no representa ningún inconveniente, el problema surge cuando estas computadoras se utilizan como **servidores** para alojar productos o soluciones de empresas.

Para las empresas de infraestructura (por ejemplo: AWS) es un problema tener recursos sin usarse porque se pierden de ganar dinero, es como para una inmobiliaria tener un departamento sin alquilar.

Virtualización

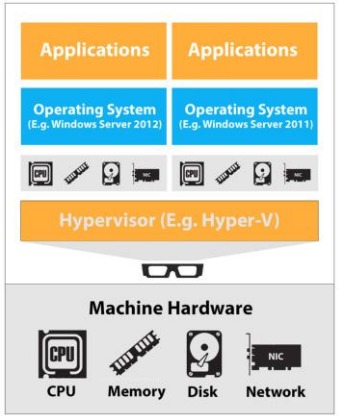
Entonces surge la virtualización, lo cual significa que en una computadora o servidor físico pueden coexistir múltiples **máquinas virtuales**, cada una con su sistema operativo y porción de hardware.



Virtualización

La virtualización a través de máquinas virtuales (VM) es algo que se sigue utilizando actualmente y durante mucho tiempo fue el único método de virtualización.

La desventaja de las VM es que cada instancia tiene su propio sistema operativo, por lo que las **imágenes** pueden presentar un tamaño considerablemente grande.



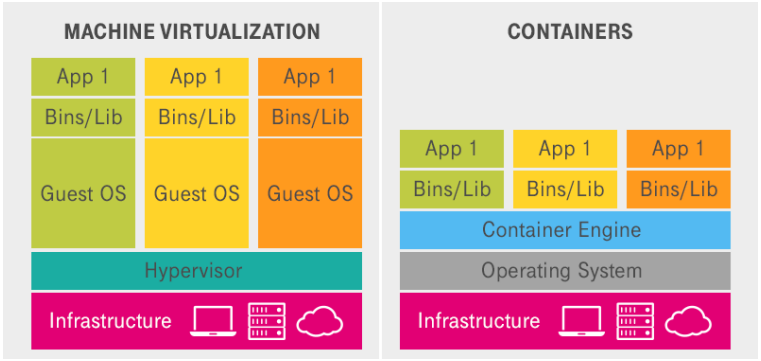
Ejemplo: Proxmox

https://www.proxmox.com/en/products/proxmox-virtual-environment/overview

Virtualización

Para subsanar esta cuestión surge Docker, implementando la virtualización a través de contenedores.

En este caso, se comparte el sistema operativo entre los diferentes containers, por lo cual las imágenes tienen un tamaño más reducido en comparación a las VM ya que es posible ejecutar aplicaciones en entornos **aislados** sin necesidad de duplicar el SO en cada uno.

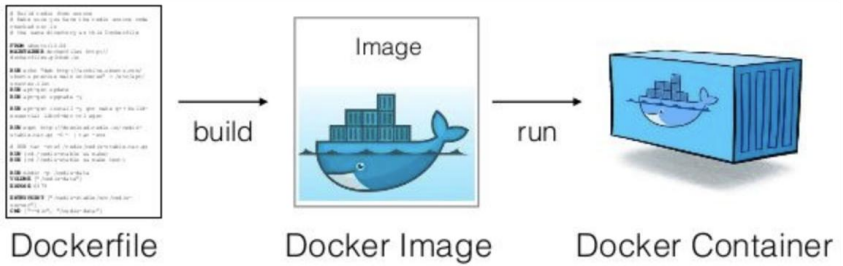


Docker

En Docker existen imágenes que se instancian en contenedores y permiten encapsular aplicaciones para su implementación, en nuestro caso las aplicaciones son modelos.

En general, en estos contenedores se instancian APIs para consumir los modelos implementados a través de requests HTTP, desde una aplicación web o mobile, por ejemplo.

Para quitarle complejidad a la implementación, vamos a hacer las estimaciones desde archivos locales **montados** como volúmenes en el container y los resultados los vamos a escribir también en archivos locales.



Docker

Para crear una imagen de docker se utiliza un archivo Dockerfile con las instrucciones para hacerlo.

En general para machine learning:

1. Se toma como base una imagen base de Python
2. Instalar librerias
    - Es indispensable únicamente incluir librerías necesarias para la ejecución del código, de lo contrario el tamaño de la imagen se verá incrementado. Para esto pueden usar entornos virtuales de Python.
3. Copiar scripts de inferencia
4. Ejecutar scripts de inferencia
    - En una implementación real se inicializa un API, en este caso procesamos los archivos

Docker

Los binarios del modelo se pueden encapsular en la imagen o montar como archivos. La primera opción es más sencilla, pero incrementa el peso de la imagen y quita flexibilidad, ya que para modificar el binario se requiere construir nuevamente la imagen.

El objetivo es que en el repositorio del TP incluyan un Dockerfile, archivo con librerías requeridas, scripts de inferencia y binarios de modelo.

Ademas, indiquen las especificaciones para ejecutar el contenedor y realizar la inferencia, así los docentes podemos clonar el repositorio y ejecutar el container localmente.

Una de las mayores ventajas que tiene Docker es que permite eliminar la diferencia de compatibilidad entre diferentes entornos de ejecución.

Docker: instalar

https://docs.docker.com/desktop/install/windows-install/

Dockerfile: ejemplo

https://hub.docker.com/_/python

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt ./

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD [ "python", "./inference.py" ]
```

Docker: comandos

https://hub.docker.com/_/python

Buildear imagen:

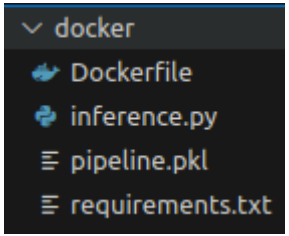
- `docker build -t inference-tp2-mlops .`

Ejecutar imagen buildeada con la carpeta files montada como volumen para leer inputs y escribir predicciones:

- `docker run -it --rm --name inference-tp2-mlops-container -v "./path/en/host/files:/path/en/container/files" inference-tp2-mlops`

Docker: ejemplo estructura

La carpeta docker dentro del repositorio debería quedar **similar** al siguiente ejemplo:



Recordar únicamente **instalar las librerías requeridas para la inferencia** (con sus respectivas versiones) y **copiar los archivos necesarios solamente** para evitar incrementar el tamaño de la imagen (NO copiar el entorno virtual)